To make the test pass, you need to add a bit of logic to the controller action:

security/05/app/controllers/tasks_controller.rb

```
def create
  @project = Project.find(params[:task][:project_id])
  unless current_user.can_view?(@project)
    redirect_to new_user_session_path
    return
  end
  @project.tasks.create(
    task_params.merge(project_order: @project.next_task_order))
  redirect_to(@project)
end
```

The new part is the unless statement, which checks to see if the current_user can see the project in question. You can trust the current_user value because it is not dependent on any data coming from the user; it's managed by the Rails session.

There are a lot of possibilities from here. You could extract the current controller method to a CreatesTask action item, which would make it easier to separate the access logic from the rest of the code. You could also add similar protection to the update, up, and down methods, which involves the design question of modeling access control from the task's perspective.

Mass Assignment Testing

Mass assignment is a common Rails security issue, caused by Rails's ability to save an arbitrary hash of attribute names and values to an instance by sending an entire hash as a parameter, as in new(params[:user]), create(params[:user]), or update_attributes(params[:user]). The security issue happens when somebody hacks a request and adds unexpected attributes to the incoming parameters, typically an attribute that you wouldn't want an arbitrary user to be able to change, such as User#admin or Project#public. (GitHub was famously hacked via this vector by a user who added himself as a committer to the Rails repo.)

Rails 4 added the concept of *strong parameters* to allow you to identify parts of the parameter hash from an incoming request as required or permissible. To be used in a mass assignment, the attributes need to be identified using the require or permit methods of the Rails parameter object. Attributes that aren't whitelisted aren't passed on to the ActiveRecord object, and they are helpfully listed in the Rails log as a warning and to make debugging these issues easier.

The Gatherer application currently uses strong parameters in one location, TasksController#update, where there is a method that defines them: